Fase 4 — Base de datos: entidades, atributos, claves, relaciones, normalización, MER y modelo lógico

Proyecto: Sistema de Reservas de Restaurante

Aprendices: Luis Eduardo Buitrago Pascuas, Laura Daniela Aviles Perdomo, Andrés Camilo Pérez Mendoza

Programa: Análisis y Desarrollo de Software (3239137) — SENA, Neiva, 2026

Motor de base de datos: MySQL 8.0 (InnoDB, utf8mb4)

1. Introducción

Este documento reúne el modelado de la base de datos del Sistema de Reservas de Restaurante. Se construye a partir de la Fase 1 (actores, épicas e historias HU-01 a HU-12), la Fase 2 (RF-01 a RF-20, RNF-01 a RNF-08, RN-01 a RN-14 y restricciones RT-01 a RT-06) y la Fase 3 (modelado UML, en especial el diagrama de clases de la Figura 2).

Se desarrollan, en este orden: identificación de entidades, definición de atributos, claves primarias y foráneas, relaciones, normalización, modelo entidad-relación (MER) y modelo lógico. El alcance se limita a clientes, mesas, disponibilidad, reservas, autenticación, roles y permisos; no se incluyen pagos, inventario ni domicilios (RT-05). El script SQL y la lógica de validación se implementan en la etapa siguiente.

1.1 Hallazgos al revisar las Fases 1, 2 y 3

Cód.	Hallazgo	Fuente	Solución adoptada
H-1	La Fase 3 dibuja Usuario → Rol como 1 a 1..*, pero modela tres subclases excluyentes de Usuario, y RF-15 y HU-09 hablan de «el rol» en singular.	Fase 3 (Fig. 2); RF-15; HU-09	Cada usuario tiene un único rol y solo puede ingresar al sistema con ese rol (Rol 1 — Usuario 0..*). Si el equipo exigiera varios roles, se cambia `usuarios.id_rol` por una tabla `usuarios_roles` sin afectar el resto.

Cód.	Hallazgo	Fuente	Solución adoptada
H-2	Rol → Permiso se dibuja 1 a 0..*, lo que implicaría que cada permiso pertenece a un solo rol; pero HU-08 restringe mesas y panel del día a dos roles a la vez.	Fase 3 (Fig. 2); HU-08	Relación muchos a muchos con la tabla `roles_permisos`, coherente con la agregación de la Fase 3.
H-3	«Reserva activa» no está definida: HU-03 habla de reserva «confirmada» y HU-06, RN-01 y RF-09 de reserva «activa».	HU-03; HU-06; RN-01; RF-09	Reserva activa = estado PENDIENTE o CONFIRMADA. CANCELADA libera la mesa (RN-07) y FINALIZADA ya concluyó (Fase 3, §8).
H-4	HU-09 exige bloqueo temporal tras intentos fallidos (la Fase 3, §7.2, muestra un contador de intentos), pero la clase Usuario no tiene atributos para ello.	HU-09; Fase 3 §7.2	Se agregan `intentos_fallidos` y `bloqueado_hasta` a `usuarios`. El token temporal de RF-20 (Fase 3, §7.4) no se guarda en este modelo; se define en la implementación.
H-5	La Fase 3 no tiene estado del usuario y hoy no es posible inactivar clientes. El administrador gestiona usuarios (Fase 1, §3) y el historial debe conservarse (HU-11, RN-08).	Fase 1 §3; HU-11; RN-08; observación del profesor	Se agrega `usuarios.estado` ENUM('ACTIVO','INACTIVO'), por defecto 'ACTIVO'. Un usuario INACTIVO no inicia sesión y no se borra, así se conserva su historial. No confundir con «reserva activa» (H-3).
H-6	RegistroCambio solo guarda responsable, fecha y descripción; RF-14 pide registrar cambios de roles y permisos, pero no a quién se le hicieron.	RF-14; RN-12; Fase 3 (Fig. 2)	Se agrega `id_usuario_afectado`, opcional. El detalle del cambio queda en `descripcion`.
H-7	La notificación es 0..1 por reserva, pero la Fase 3, §7.12, menciona reintentos de envío.	Fase 3 (Fig. 2) y §7.12	Se mantiene 0..1; un reintento actualiza `estado_envio` de la misma notificación.

Cód.	Hallazgo	Fuente	Solución adoptada
H-8	ConfiguracionRestaurante no tiene HU ni RF propia (Fase 3, §4.4 y §5.1, sujeta a RT-05), pero RN-05, RN-06 y RF-05 dependen de «la configuración del restaurante».	Fase 3 §5.1; RN-05; RN-06	Se incorpora como tabla de fila única. Si no se aprueba la extensión, se elimina sin afectar el resto.
H-9	El estado de la mesa solo contempla LIBRE, RESERVADA y OCUPADA; no hay forma de sacar una mesa de servicio.	HU-02; RF-02; Fase 3 (EstadoMesa); observación del profesor	Se amplían los estados con INACTIVA y MANTENIMIENTO. Una mesa en esos estados no se ofrece en la consulta de disponibilidad ni recibe reservas nuevas.

Los puntos H-4, H-5, H-6, H-8 y H-9 son extensiones al modelo de la Fase 3 aprobadas por el equipo dentro de este documento (H-5 y H-9 por observación del profesor); los demás son ajustes de multiplicidad o interpretación.

1.2 Decisiones de diseño

Cód.	Decisión	Justificación
D-1	Se convierten en tablas Usuario, Rol, Permiso, Mesa, Reserva, Notificación, RegistroCambio y ConfiguracionRestaurante.	Cada una tiene identidad, atributos y ciclo de vida propios en la Fase 3.
D-2	Cliente, Recepcionista y Administrador no tienen tabla propia; el tipo de usuario lo define su rol.	En la Fase 3 las subclases solo agregan métodos, ningún atributo.
D-3	EstadoMesa y EstadoReserva se implementan como columnas ENUM, equivalentes a un CHECK sobre la lista de valores. EstadoMesa se amplía con INACTIVA y MANTENIMIENTO (H-9).	La Fase 3 (§5.1) las define como enumeraciones sin atributos ni comportamiento.
D-4	Usuario–Rol es 1:N: cada usuario tiene un único rol y solo accede con él. Rol–Permiso es N:M (H-1, H-2).	Ver hallazgos.

Cód.	Decisión	Justificación
D-5	La asociación «confirma/reubica» entre Recepcionista y Reserva no se convierte en tabla.	Es una capacidad del rol; confirmar cambia `reservas.estado` y reubicar cambia `reservas.id_mesa`. Ningún RF ni RN exige guardar qué recepcionista la hizo.
D-6	`configuracion_restaurante` es una tabla de una sola fila, sin clave foránea desde `reservas`.	Los parámetros (RN-05, RN-06) son del restaurante, no de cada reserva; se aplican al crear o modificar la reserva y así no se repiten en cada fila.
D-7	Los nombres de columna conservan los atributos de la Fase 3 en snake_case. Las columnas nuevas son claves foráneas y las de H-4, H-5 y H-6.	Mantener coherencia con los documentos anteriores.
D-8	Motor MySQL 8.0 con InnoDB (transacciones y claves foráneas) y utf8mb4.	RT-03 exige una base de datos que mantenga la integridad de las reservas.
D-9	No se eliminan físicamente usuarios, mesas ni reservas con historial: los usuarios se inactivan (estado INACTIVO), las mesas salen de servicio (INACTIVA) y las claves foráneas usan RESTRICT.	HU-11, RN-08.
D-10	Los nombres de tabla van en plural: roles, permisos, roles_permisos, usuarios, mesas, reservas, notificaciones y registros_cambio. `configuracion_restaurante` queda en singular por ser de una sola fila. Las columnas siguen en singular.	Convención de nomenclatura solicitada por el profesor. Las entidades del MER conservan su nombre singular.

2. Objetivos

Objetivo general: diseñar la estructura de datos del Sistema de Reservas de Restaurante, normalizada y coherente con las Fases 1, 2 y 3, de modo que soporte las historias de usuario y las reglas de negocio del MVP.

Objetivos específicos:

- Identificar las entidades, sus atributos, claves y relaciones a partir de las fases anteriores.

- Normalizar las estructuras hasta la tercera forma normal, evitando redundancia y anomalías.
- Representar las entidades y sus relaciones en un modelo entidad-relación con cardinalidad y participación.
- Transformar el MER en un modelo lógico relacional para MySQL, indicando qué reglas de negocio garantiza la base de datos y cuáles requieren lógica adicional.

3. Identificación de entidades

Se parte de las clases del diagrama de clases de la Fase 3. La generalización Usuario → Cliente / Recepcionista / Administrador se resuelve con una sola tabla `usuarios` y el rol asignado (D-2).

Entidad (tabla)	Descripción	Origen
usuarios	Persona con acceso al sistema (cliente, recepcionista o administrador), con estado ACTIVO o INACTIVO	Clase Usuario y subclases; HU-01, HU-09, HU-10
roles	Perfil de acceso: CLIENTE, RECEPCIONISTA, ADMINISTRADOR	Clase Rol; HU-08
permisos	Acción que puede ejecutar un rol	Clase Permiso; RF-13
roles_permisos	Tabla intermedia: permisos de cada rol (N:M)	Rol agrupa Permiso
mesas	Mesa con número, capacidad, ubicación y estado (incluye fuera de servicio)	Clase Mesa; HU-02, RF-02
reservas	Reserva de una mesa por un cliente en una fecha y horario	Clase Reserva; HU-04 a HU-07
notificaciones	Mensaje enviado al cliente al crear su reserva	Clase Notificacion; RF-06
registros_cambio	Auditoría de cambios de rol y permisos	Clase RegistroCambio; RN-12, RF-14

Entidad (tabla)	Descripción	Origen
configuracion_restaurante	Parámetros generales: anticipación mínima, estado inicial y horario	Clase ConfiguracionRestaurante; RN-05, RN-06

Resultado: ocho entidades y una tabla intermedia (`roles_permisos`), nueve tablas en total. Los métodos de las clases (por ejemplo validarDisponibilidad()) son lógica de la aplicación y no hacen parte del modelo de datos.

4. Definición de atributos

NN = no nulo, UQ = único, PK = clave primaria, FK = clave foránea, CK = restricción CHECK.

usuarios

Atributo	Tipo MySQL	Restricciones	Descripción
id_usuario	INT	PK, AUTO_INCREMENT	Identificador
nombre	VARCHAR(100)	NN, no vacío	Nombre completo (HU-01)
correo	VARCHAR(120)	NN, UQ, formato válido	Correo electrónico (RN-04)
telefono	VARCHAR(20)	NN, UQ, CK de 7 a 20 caracteres	Teléfono (RN-04)
contrasena_hash	VARCHAR(255)	NN	Hash de la contraseña, nunca texto plano
intentos_fallidos	SMALLINT UNSIGNED	NN, DEFAULT 0	Contador para el bloqueo temporal (HU-09)
bloqueado_hasta	DATETIME	NULL	Fin del bloqueo temporal

Atributo	Tipo MySQL	Restricciones	Descripción
estado	ENUM('ACTIVO','INACTIVO')	NN, DEFAULT 'ACTIVO'	Estado de la cuenta; INACTIVO impide el acceso y conserva el historial (H-5)
fecha_registro	DATE	NN, DEFAULT (CURRENT_DATE)	Fecha de registro
id_rol	INT	NN, FK → roles	Único rol del usuario, con el que inicia sesión (H-1)

roles: id_rol INT PK AUTO_INCREMENT · nombre VARCHAR(30) NN UQ.

permisos: id_permiso INT PK AUTO_INCREMENT · nombre VARCHAR(60) NN UQ · descripcion VARCHAR(200) NULL.

roles_permisos: id_rol INT FK → roles · id_permiso INT FK → permisos (clave primaria compuesta).

mesas

Atributo	Tipo MySQL	Restricciones	Descripción
id_mesa	INT	PK, AUTO_INCREMENT	Identificador
numero	INT	NN, UQ, CK > 0	Número de mesa (RN-03)
capacidad	INT	NN, CK > 0	Personas que admite (HU-02)
ubicacion	VARCHAR(50)	NN	Zona del restaurante (RF-02)
estado	ENUM('LIBRE','RESERVADA','OCUPADA','INACTIVA','MANTENIMIENTO')	NN, DEFAULT 'LIBRE'	Estado de la mesa; INACTIVA y MANTENIMIENTO = fuera de servicio (H-9)

reservas

Atributo	Tipo MySQL	Restricciones	Descripción
id_reserva	INT	PK, AUTO_INCREMENT	Identificador interno
codigo_unico	VARCHAR(25)	NN, UQ, no modificable	Código visible para el cliente (RF-05)
id_cliente	INT	NN, FK → usuarios	Cliente que reserva

Atributo	Tipo MySQL	Restricciones	Descripción
id_mesa	INT	NN, FK → mesas	Mesa asignada
fecha	DATE	NN	Día de la reserva
hora_inicio	TIME	NN	Hora de inicio
hora_fin	TIME	NN, CK > hora_inicio	Hora de fin
numero_personas	INT	NN, CK > 0	Comensales (RN-02)
estado	ENUM('PENDIENTE','CONFIRMADA','CANCELADA','FINALIZADA')	NN	Estado de la reserva
fecha_creacion	DATETIME	NN, DEFAULT CURRENT_TIMESTAMP	Momento de creación

notificaciones

Atributo	Tipo MySQL	Restricciones	Descripción
id_notificacion	INT	PK, AUTO_INCREMENT	Identificador
id_reserva	INT	NN, UQ, FK → reservas	Reserva que la origina
tipo	VARCHAR(30)	NN	Tipo de mensaje
mensaje	VARCHAR(300)	NN	Texto enviado
fecha_envio	DATETIME	NULL	Momento del envío
estado_envio	ENUM('PENDIENTE','ENVIADA','FALLIDA')	NN, DEFAULT 'PENDIENTE'	Resultado del envío (Fase 3, §7.12)

registros_cambio

Atributo	Tipo MySQL	Restricciones	Descripción
id_registro	INT	PK, AUTO_INCREMENT	Identificador
id_usuario_responsable	INT	NN, FK → usuarios	Quien hizo el cambio (RN-12)
id_usuario_afectado	INT	NULL, FK → usuarios	A quién se le cambió el rol o los permisos (H-6)
descripcion	VARCHAR(300)	NN	Detalle del cambio

Atributo	Tipo MySQL	Restricciones	Descripción
fecha_hora	DATETIME	NN, DEFAULT CURRENT_TIMESTAMP	Cuándo se hizo (RF-14)

configuracion_restaurante (una sola fila)

Atributo	Tipo MySQL	Restricciones	Descripción
id_configuracion	TINYINT	PK, CK = 1	Fila única
anticipacion_minima_minutos	INT	NN, CK ⩾ 0, DEFAULT 60	Anticipación mínima (RN-05)
estado_inicial_reserva	ENUM('PENDIENTE','CONFIRMADA')	NN	Estado al crear la reserva (RN-06)
horario_apertura	TIME	NN	Apertura del restaurante
horario_cierre	TIME	NN, CK > horario_apertura	Cierre del restaurante

5. Definición de claves primarias y foráneas

Todas las claves primarias son sustitutas (AUTO_INCREMENT), salvo las tablas intermedias, que usan clave compuesta. Los identificadores naturales (correo, teléfono, número de mesa, código de reserva) se mantienen como claves candidatas con restricción UNIQUE.

Claves primarias y candidatas

Tabla	Clave primaria	Claves candidatas (UNIQUE)
usuarios	id_usuario	correo, telefono
roles	id_rol	nombre
permisos	id_permiso	nombre
roles_permisos	(id_rol, id_permiso)	—
mesas	id_mesa	numero
reservas	id_reserva	codigo_unico

Tabla	Clave primaria	Claves candidatas (UNIQUE)
notificaciones	id_notificacion	id_reserva
registros_cambio	id_registro	—
configuracion_restaurante	id_configuracion (siempre 1)	—

Claves foráneas. Todas usan ON UPDATE CASCADE; la última columna indica qué pasa al borrar el registro referenciado (D-9).

Tabla	Clave foránea	Referencia	Al borrar
usuarios	id_rol	roles(id_rol)	RESTRICT
roles_permisos	id_rol	roles(id_rol)	CASCADE
roles_permisos	id_permiso	permisos(id_permiso)	CASCADE
reservas	id_cliente	usuarios(id_usuario)	RESTRICT
reservas	id_mesa	mesas(id_mesa)	RESTRICT
notificaciones	id_reserva	reservas(id_reserva)	CASCADE
registros_cambio	id_usuario_responsable	usuarios(id_usuario)	RESTRICT
registros_cambio	id_usuario_afectado	usuarios(id_usuario)	SET NULL

6. Definición de relaciones

Relación	Cardinalidad	Implementación
Rol asigna Usuario	rol 1 — 0..N usuario	usuarios.id_rol
Rol agrupa Permiso	rol N — M permiso	tabla intermedia roles_permisos
Cliente realiza Reserva	usuario 1 — 0..N reserva	reservas.id_cliente
Mesa se asigna a Reserva	mesa 1 — 0..N reserva	reservas.id_mesa
Reserva genera Notificación	reserva 1 — 0..1 notificación	notificaciones.id_reserva (UQ)
Usuario es responsable de RegistroCambio	usuario 1 — 0..N registro de cambio	registros_cambio.id_usuario_responsable

Relación	Cardinalidad	Implementación
Usuario es afectado por RegistroCambio	usuario 0..1 — 0..N registro de cambio	registros_cambio.id_usuario_afectado

`configuracion_restaurante` parametriza las reservas de forma lógica, pero no lleva clave foránea desde `reservas`: es una fila única cuyos valores se aplican al crear o modificar una reserva (D-6). La asociación «confirma/reubica» entre Recepcionista y Reserva tampoco se persiste (D-5).

7. Normalización

7.1 Para qué sirve en este proyecto

La normalización organiza los datos para que cada uno se guarde en un solo lugar y dependa de la clave correcta (1FN, 2FN y 3FN). Importa aquí por tres razones: la Fase 1 identifica que clientes y reservas «regados en varios lugares» causan errores; HU-10 exige que al cambiar teléfono o correo el cambio se refleje en las reservas del cliente sin tocar su historial; y RN-03 y RN-04 exigen unicidad de número de mesa, teléfono y correo, mucho más fácil de controlar si cada dato existe una sola vez.

7.2 Estructuras iniciales

Se parte de dos relaciones de trabajo que agrupan los atributos de la Fase 3 como lo haría una hoja de control manual. No son tablas finales.

RESERVA_GENERAL (clave de trabajo: id_reserva)

Grupo	Atributos
Reserva	id_reserva, codigo_unico, fecha, hora_inicio, hora_fin, numero_personas, estado_reserva, fecha_creacion
Cliente	cliente_nombre, cliente_correo, cliente_telefono, cliente_contrasena_hash, cliente_estado, cliente_fecha_registro
Mesa	mesa_numero, mesa_capacidad, mesa_ubicacion, mesa_estado
Notificación	notif_tipo, notif_mensaje, notif_fecha_envio, notif_estado_envio

Grupo	Atributos
Configuración	anticipacion_minima_minutos, estado_inicial_reserva, horario_apertura, horario_cierre

USUARIO_GENERAL (clave de trabajo: id_usuario)

Grupo	Atributos
Usuario	id_usuario, nombre, correo, telefono, contrasena_hash, intentos_fallidos, bloqueado_hasta, estado, fecha_registro
Rol y permisos	rol, permisos_del_rol (lista)
Registro de cambios	cambios_realizados (lista)

Ejemplo con datos ficticios de RESERVA_GENERAL (solo algunas columnas):

id_reserva	codigo_unico	fecha	hora_inicio	cliente_nombre	cliente_telefono	mesa_numero	mesa_capacidad
1	RES-001	2026-10-20	19:00	Ana Gómez	3101234567	5	4
2	RES-002	2026-10-22	20:00	Ana Gómez	3101234567	5	4
3	RES-003	2026-10-22	13:00	Luis Rojas	3207654321	2	2

Se observa de inmediato que el teléfono de Ana y la capacidad de la mesa 5 se repiten en cada fila.

7.3 Dependencias funcionales consideradas

Solo se usan dependencias derivadas de las claves de la Fase 3 o de las reglas de unicidad documentadas.

Entidad	Dependencias funcionales	Origen
Usuario	id_usuario → nombre, correo, telefono, contrasena_hash, intentos_fallidos, bloqueado_hasta, estado, fecha_registro, id_rol; correo → id_usuario; telefono → id_usuario	Fig. 2; RN-04; H-4, H-5
Rol	id_rol → nombre	Fig. 2

Entidad	Dependencias funcionales	Origen
Permiso	id_permiso → nombre, descripcion	Fig. 2
Mesa	id_mesa → numero, capacidad, ubicacion, estado; numero → id_mesa	Fig. 2; RN-03
Reserva	id_reserva → codigo_unico, fecha, hora_inicio, hora_fin, numero_personas, estado, fecha_creacion, id_cliente, id_mesa; codigo_unico → id_reserva	Fig. 2; RF-05
Notificación	id_notificacion → tipo, mensaje, fecha_envio, estado_envio, id_reserva	Fig. 2
RegistroCambio	id_registro → id_usuario_responsable, id_usuario_afectado, fecha_hora, descripcion	Fig. 2; RN-12
ConfiguracionRestaurante	id_configuracion → anticipacion_minima_minutos, estado_inicial_reserva, horario_apertura, horario_cierre	Fig. 2

No se asume que (id_mesa, fecha, hora_inicio) determine una reserva: entre reservas activas no puede repetirse (RN-01), pero una reserva cancelada puede coincidir con una nueva (RN-07, RN-08), por lo que no es una dependencia funcional válida ni una clave candidata.

7.4 Primera forma normal (1FN)

La 1FN exige valores atómicos y ausencia de grupos repetitivos.

Atributo o grupo	¿Cumple?	Observación
nombre, correo, telefono	Sí	Un solo valor cada uno. `nombre` se mantiene como un atributo porque así lo definen las Fases 1 y 3 y HU-12 busca por nombre.
fecha, hora_inicio, hora_fin	Sí	Fecha y horas en atributos separados.
ubicacion, estado	Sí	Un solo valor por mesa, reserva o usuario.
Datos de notificación en RESERVA_GENERAL	Sí	La relación es 0..1, no hay lista.

Atributo o grupo	¿Cumple?	Observación
permisos_del_rol	**No**	Un rol agrupa 0..* permisos: varios valores en una celda.
cambios_realizados	**No**	Un usuario genera 0..* registros de cambio: grupo repetitivo.

Ejemplo del problema en USUARIO_GENERAL (permisos tomados del criterio de HU-08):

id_usuario	nombre	rol	permisos_del_rol
2	Usuario de ejemplo	Recepcionista	gestionar mesas, ver panel de reservas del día

Ajuste: cada elemento de la lista pasa a una fila propia.

- USUARIO_1FN(**id_usuario**, nombre, correo, telefono, contrasena_hash, intentos_fallidos, bloqueado_hasta, estado, fecha_registro, id_rol, nombre_rol)
- ROL_PERMISO_1FN(**id_rol, id_permiso**, nombre_rol, nombre_permiso, descripcion_permiso)
- REGISTRO_CAMBIO(**id_registro**, id_usuario_responsable, id_usuario_afectado, fecha_hora, descripcion)

RESERVA_GENERAL ya cumple la 1FN sin cambios.

7.5 Segunda forma normal (2FN)

La 2FN exige que, en tablas con clave compuesta, cada atributo dependa de la clave completa. Solo ROL_PERMISO_1FN tiene clave compuesta; las demás usan una clave de un solo atributo, por lo que no pueden tener dependencias parciales.

Dependencias parciales en ROL_PERMISO_1FN: id_rol → nombre_rol e id_permiso → nombre_permiso, descripcion_permiso.

Problemas que causa: para renombrar un permiso habría que modificar todas las filas donde aparece (actualización); no se podría registrar un permiso nuevo sin asignarlo a un rol (inserción); y al quitar el último permiso de un rol se perdería el nombre del rol (eliminación).

Ajuste: se separan los datos de cada clave.

- ROL(**id_rol**, nombre)
- PERMISO(**id_permiso**, nombre, descripcion)

- ROL_PERMISO(**id_rol, id_permiso**), ambas columnas claves foráneas, sin atributos adicionales, por lo que cumple la 2FN.

7.6 Tercera forma normal (3FN)

La 3FN exige que no existan dependencias transitivas: un atributo no clave no debe depender de otro atributo no clave.

En RESERVA_GENERAL (clave id_reserva):

- id_reserva → id_cliente → cliente_nombre, cliente_correo, cliente_telefono, cliente_estado, …
- id_reserva → id_mesa → mesa_numero, mesa_capacidad, mesa_ubicacion, mesa_estado
- Los parámetros de configuración no dependen de la reserva sino del restaurante: son idénticos en todas las filas. No es una dependencia transitiva por clave foránea, pero causa la misma anomalía de actualización (cambiar la anticipación mínima obligaría a modificar todas las reservas), por eso se separan en una tabla de una sola fila (D-6).

En USUARIO_1FN: id_usuario → id_rol → nombre_rol. El nombre ya está en ROL, así que se elimina `nombre_rol` y queda solo `id_rol` como clave foránea.

Anomalías en RESERVA_GENERAL antes del ajuste:

Tipo	Ejemplo con el sistema de reservas	Cómo se resuelve
Inserción	No se puede registrar una mesa nueva (HU-02) ni un cliente nuevo (HU-01) hasta que exista una reserva que los incluya.	Mesa y usuario son tablas independientes.
Actualización	Si Ana cambia su teléfono (HU-10) hay que modificar todas sus filas; si se olvida una, queda con dos teléfonos y se debilita RN-04. Lo mismo al cambiar la capacidad de la mesa 5.	El teléfono vive solo en `usuarios` y la capacidad solo en `mesas`; la reserva apunta por clave foránea.
Eliminación	Al borrar la única reserva de una mesa o de un cliente se pierden también los datos de esa mesa o de ese cliente.	Los datos de mesa y cliente no dependen de que exista una reserva.

Al dejar solo `id_cliente` en la reserva, el cambio de datos de contacto se refleja de inmediato en las reservas del cliente sin alterar el historial, como pide HU-10.

Notificación. Sus atributos dependen solo de id_notificacion. Dejarlos dentro de Reserva tampoco rompería la 3FN (la relación es 0..1); se mantiene como tabla propia por la composición de la Fase 3: evita columnas vacías en las reservas sin notificación y separa el ciclo de vida del envío.

Estado de la mesa. `mesas.estado` depende de id_mesa, así que no viola la 3FN. Podría deducirse de las reservas, pero HU-02, RF-02 y RNF-02 piden almacenarlo y mantenerlo actualizado; debe modificarse en la misma operación que cambia la reserva. Los valores INACTIVA y MANTENIMIENTO no se deducen de ninguna reserva: son una decisión del personal, por lo que el dato debe guardarse.

Estado del usuario. `usuarios.estado` depende solo de id_usuario; es un dato propio de la cuenta y no rompe la 3FN.

7.7 Resultado final de la normalización

Tabla	Clave primaria	Forma normal	Justificación
usuarios	id_usuario	3FN	Todos los atributos dependen de la clave; el nombre del rol se obtiene por id_rol. Correo y teléfono son claves candidatas (RN-04).
roles	id_rol	3FN	Un solo atributo descriptivo.
permisos	id_permiso	3FN	Nombre y descripción dependen solo de la clave.
roles_permisos	(id_rol, id_permiso)	3FN	Tabla de asociación sin atributos propios.
mesas	id_mesa	3FN	Todos los atributos describen a la mesa; numero es clave candidata (RN-03).
reservas	id_reserva	3FN	Datos propios de la reserva y claves foráneas hacia cliente y mesa; codigo_unico es clave candidata (RF-05).
notificaciones	id_notificacion	3FN	Atributos del envío; id_reserva identifica la reserva de origen (0..1).

Tabla	Clave primaria	Forma normal	Justificación
registros_cambio	id_registro	3FN	Guarda usuarios solo como claves foráneas, sin copiar sus datos.
configuracion_restaurante	id_configuracion	3FN	Parámetros generales sin dependencias entre sí.

El resultado son nueve tablas en 3FN. Ninguna repite datos de otra: cada hecho se guarda una sola vez y las relaciones se expresan con claves foráneas. Roles, permisos, mesas, notificaciones, registros de cambio y configuración ya cumplían las tres formas normales y no requirieron ajustes.

8. Modelo entidad-relación (MER)

8.1 Entidades del MER

A partir de las decisiones D-1 a D-6, las entidades del MER son ocho. Las entidades llevan nombre singular; al pasar al modelo lógico cada una se convierte en una tabla en plural (D-10). La relación muchos a muchos entre Rol y Permiso se muestra directamente en el MER y se resuelve con una tabla intermedia en el modelo lógico (sección 9). CONFIGURACION_RESTAURANTE es una entidad de parámetros sin relación con las demás (D-6).

Entidad	Clase de la Fase 3	Clave primaria	Atributos principales
USUARIO	Usuario (con Cliente, Recepcionista y Administrador)	id_usuario	nombre, correo, telefono, contrasena_hash, intentos_fallidos, bloqueado_hasta, estado (ACTIVO o INACTIVO), fecha_registro
ROL	Rol	id_rol	nombre
PERMISO	Permiso	id_permiso	nombre, descripcion
MESA	Mesa (con EstadoMesa)	id_mesa	numero, capacidad, ubicacion, estado (LIBRE, RESERVADA, OCUPADA, INACTIVA o MANTENIMIENTO)
RESERVA	Reserva (con EstadoReserva)	id_reserva	codigo_unico, fecha, hora_inicio, hora_fin, numero_personas, estado, fecha_creacion
NOTIFICACION	Notificacion	id_notificacion	tipo, mensaje, fecha_envio, estado_envio
REGISTRO_CAMBIO	RegistroCambio	id_registro	descripcion, fecha_hora

Entidad	Clase de la Fase 3	Clave primaria	Atributos principales
CONFIGURACION_RESTAURANTE	ConfiguracionRestaurante	id_configuracion	anticipacion_minima_minutos, estado_inicial_reserva, horario_apertura, horario_cierre

Cliente, Recepcionista y Administrador se representan por el rol del usuario. EstadoMesa y EstadoReserva (PENDIENTE, CONFIRMADA, CANCELADA, FINALIZADA) son valores permitidos de las columnas `estado`, no entidades.

8.2 Diagrama entidad-relación

Notación de patas de gallo. El diagrama muestra solo los atributos propios de cada entidad; las claves foráneas aparecen en el modelo lógico (sección 9.4). El código puede copiarse en mermaid.live para exportarlo como imagen.

```
erDiagram
    ROL ||--o{ USUARIO : asigna
    ROL }o--o{ PERMISO : agrupa
    USUARIO ||--o{ RESERVA : realiza
    MESA ||--o{ RESERVA : se_asigna
    RESERVA ||--o| NOTIFICACION : genera
    USUARIO ||--o{ REGISTRO_CAMBIO : responsable_de
    USUARIO |o--o{ REGISTRO_CAMBIO : afectado_por

    USUARIO {
        entero id_usuario PK
        texto nombre
        texto correo UK
        texto telefono UK
        texto contrasena_hash
        entero intentos_fallidos
        fecha_hora bloqueado_hasta
        enumerado estado
        fecha fecha_registro
    }
    ROL {
        entero id_rol PK
        texto nombre UK
    }
    PERMISO {
        entero id_permiso PK
        texto nombre UK
        texto descripcion
    }
    MESA {
        entero id_mesa PK
```

```
        entero numero UK
        entero capacidad
        texto ubicacion
        enumerado estado
    }
    RESERVA {
        entero id_reserva PK
        texto codigo_unico UK
        fecha fecha
        hora hora_inicio
        hora hora_fin
        entero numero_personas
        enumerado estado
        fecha_hora fecha_creacion
    }
    NOTIFICACION {
        entero id_notificacion PK
        texto tipo
        texto mensaje
        fecha_hora fecha_envio
        enumerado estado_envio
    }
    REGISTRO_CAMBIO {
        entero id_registro PK
        texto descripcion
        fecha_hora fecha_hora
    }
    CONFIGURACION_RESTAURANTE {
        entero id_configuracion PK
        entero anticipacion_minima_minutos
        enumerado estado_inicial_reserva
        hora horario_apertura
        hora horario_cierre
    }
```

Nota: la relación USUARIO realiza RESERVA corresponde a Cliente → Reserva de la Fase 3; solo participan usuarios con rol Cliente.

8.3 Relaciones, cardinalidad y participación

Relación	Cardinalidad	Participación	Justificación
ROL — USUARIO (asigna)	Rol 1 — Usuario 0..* (1:N)	Total de Usuario (todo usuario tiene exactamente un rol); parcial de Rol	RF-13, RF-15 y HU-09 hablan del rol del usuario. Ajusta el 1..* de la Fase 3 (H-1).
ROL — PERMISO (agrupa)	Rol 0..* — Permiso 0..* (N:M)	Parcial en ambos lados	HU-08: mesas y panel del día se restringen a dos roles a la vez (H-2).
USUARIO (cliente) — RESERVA (realiza)	Usuario 1 — Reserva 0..* (1:N)	Total de Reserva; parcial de Usuario	Fase 3 (Cliente → Reserva); HU-11, RF-18, RN-10.
MESA — RESERVA (se asigna a)	Mesa 1 — Reserva 0..* (1:N)	Total de Reserva; parcial de Mesa	RN-01 y RN-02. La cardinalidad permite varias reservas por mesa; que no se crucen lo controla la validación (sección 9.7).
RESERVA — NOTIFICACION (genera)	Reserva 1 — Notificación 0..1 (1:1 opcional)	Total de Notificación	RF-06; composición de la Fase 3.
USUARIO — REGISTRO_CAMBIO (responsable)	Usuario 1 — Registro 0..* (1:N)	Total de Registro	HU-08, RF-14, RN-12.
USUARIO — REGISTRO_CAMBIO (afectado)	Usuario 0..1 — Registro 0..*	Parcial en ambos lados	H-6: no todo cambio recae sobre un usuario (por ejemplo, un cambio de permisos de un rol).

8.4 Elementos del modelo UML que no se convierten en entidad

- **Asociación Recepcionista — Reserva («confirma/reubica»):** no se persiste como tabla (D-5). Sus efectos quedan en `reservas.estado` y `reservas.id_mesa`.
- **Subclases de Usuario y enumeraciones:** explicadas en D-2 y D-3.
- **Métodos de las clases** (validarDisponibilidad(), generarCodigoUnico()): son lógica de la aplicación.

9. Modelo lógico

9.1 Convenciones y motor

El modelo lógico transforma el MER en tablas relacionales para **MySQL 8.0** con motor InnoDB (transacciones y claves foráneas) y conjunto de caracteres utf8mb4. Notación: **PK** clave primaria, **FK** clave foránea, **UK** valor único, **NN** no nulo, **CK** restricción CHECK (se aplica desde MySQL 8.0.16). Los estados se implementan como ENUM, que equivale a un CHECK sobre la lista de valores; si se prefiere VARCHAR con CHECK, la lista es la misma (por ejemplo, CHECK (estado IN ('LIBRE','RESERVADA','OCUPADA','INACTIVA','MANTENIMIENTO')) en `mesas`). BOOLEAN equivale a TINYINT(1). Los nombres de tablas van en minúscula, plural y snake_case (D-10), salvo `configuracion_restaurante`, de fila única. Los tipos, valores por defecto y restricciones columna por columna están en la sección 4 y las políticas de las claves foráneas en la sección 5.

9.2 Estrategia para la herencia de Usuario

La Fase 3 define a Usuario como clase abstracta con tres subclases. Se evaluaron tres estrategias:

Estrategia	Ventajas	Limitaciones
Tabla única (`usuarios` con `id_rol`)	Simple; el inicio de sesión consulta una sola tabla; cambiar el rol (HU-08) es actualizar una columna	La FK de `reservas` apunta a `usuarios`, no a una tabla exclusiva de clientes
Tabla por subclase	La FK de `reservas` puede apuntar solo a `clientes`	Tres tablas sin atributos propios que repiten el dato del rol; cambiar de rol obligaría a mover filas

Estrategia	Ventajas	Limitaciones
Una tabla por clase concreta	Sin uniones	Repite correo, teléfono y contraseña en tres tablas; la unicidad queda dispersa y RegistroCambio no podría apuntar a una sola tabla

Decisión: tabla única. Las subclases solo agregan métodos, así que no hay datos que separar. Limitaciones aceptadas:

- La base de datos no impide por sí sola que `reservas.id_cliente` apunte a un usuario sin rol Cliente; esa verificación queda en la validación al crear la reserva.
- La unicidad de correo y teléfono se aplica a todos los usuarios, no solo a clientes: es más estricta que RN-04, pero coherente con la Fase 3, donde esos atributos pertenecen a Usuario.
- Si algún tipo de usuario llegara a necesitar atributos propios, se agregaría una tabla de extensión para ese tipo.

9.3 Esquema relacional

```
roles(id_rol PK, nombre UK)
permisos(id_permiso PK, nombre UK, descripcion)
roles_permisos(id_rol PK/FK -> roles, id_permiso PK/FK -> permisos)
usuarios(id_usuario PK, nombre, correo UK, telefono UK, contrasena_hash,
        intentos_fallidos, bloqueado_hasta, estado, fecha_registro,
        id_rol FK -> roles)
mesas(id_mesa PK, numero UK, capacidad, ubicacion, estado)
reservas(id_reserva PK, codigo_unico UK, fecha, hora_inicio, hora_fin,
        numero_personas, estado, fecha_creacion,
        id_cliente FK -> usuarios, id_mesa FK -> mesas)
notificaciones(id_notificacion PK, id_reserva UK/FK -> reservas, tipo,
        mensaje, fecha_envio, estado_envio)
registros_cambio(id_registro PK, id_usuario_responsable FK -> usuarios,
        id_usuario_afectado FK -> usuarios, descripcion, fecha_hora)
configuracion_restaurante(id_configuracion PK, anticipacion_minima_minutos,
        estado_inicial_reserva, horario_apertura, horario_cierre)
```

9.4 Diagrama del modelo lógico

Incluye las claves foráneas, la tabla intermedia `roles_permisos` y los tipos de MySQL.

```
erDiagram
    roles ||--o{ usuarios : asigna
    roles ||--o{ roles_permisos : tiene
    permisos ||--o{ roles_permisos : pertenece_a
    usuarios ||--o{ reservas : realiza
    mesas ||--o{ reservas : se_asigna
    reservas ||--o| notificaciones : genera
    usuarios ||--o{ registros_cambio : responsable_de
    usuarios |o--o{ registros_cambio : afectado_por

    usuarios {
        int id_usuario PK
        varchar nombre
        varchar correo UK
        varchar telefono UK
        varchar contrasena_hash
        smallint intentos_fallidos
        datetime bloqueado_hasta
        enum estado
        date fecha_registro
        int id_rol FK
    }
    roles {
        int id_rol PK
        varchar nombre UK
    }
    permisos {
        int id_permiso PK
        varchar nombre UK
        varchar descripcion
    }
    roles_permisos {
        int id_rol PK, FK
        int id_permiso PK, FK
    }
    mesas {
        int id_mesa PK
        int numero UK
        int capacidad
        varchar ubicacion
        enum estado
    }
    reservas {
        int id_reserva PK
        varchar codigo_unico UK
        int id_cliente FK
```

```
        int id_mesa FK
        date fecha
        time hora_inicio
        time hora_fin
        int numero_personas
        enum estado
        datetime fecha_creacion
    }
    notificaciones {
        int id_notificacion PK
        int id_reserva FK, UK
        varchar tipo
        varchar mensaje
        datetime fecha_envio
        enum estado_envio
    }
    registros_cambio {
        int id_registro PK
        int id_usuario_responsable FK
        int id_usuario_afectado FK
        varchar descripcion
        datetime fecha_hora
    }
    configuracion_restaurante {
        tinyint id_configuracion PK
        int anticipacion_minima_minutos
        enum estado_inicial_reserva
        time horario_apertura
        time horario_cierre
    }
```

9.5 Transformación de las relaciones del MER al modelo lógico

Relación del MER	Regla aplicada	Resultado
ROL — USUARIO (1:N)	La clave del lado «uno» pasa como FK al lado «muchos»	usuarios.id_rol
USUARIO — RESERVA (1:N)	Igual	reservas.id_cliente
MESA — RESERVA (1:N)	Igual	reservas.id_mesa

Relación del MER	Regla aplicada	Resultado
USUARIO — REGISTRO_CAMBIO (1:N, dos relaciones)	Igual; la segunda es opcional	registros_cambio.id_usuario_responsable e id_usuario_afectado
RESERVA — NOTIFICACION (1:0..1)	FK en el lado dependiente con restricción de unicidad	notificaciones.id_reserva (UK, NN)
ROL — PERMISO (N:M)	Tabla intermedia con las dos claves	roles_permisos(id_rol, id_permiso)
Herencia de Usuario	Tabla única	usuarios con id_rol (9.2)
Enumeraciones de estado	Columna ENUM	usuarios.estado, mesas.estado, reservas.estado, notificaciones.estado_envio
Configuración del restaurante	Tabla de fila única, sin FK	configuracion_restaurante (D-6)
Nombres de tabla	Plural (D-10)	roles, permisos, roles_permisos, usuarios, mesas, reservas, notificaciones, registros_cambio

La única relación muchos a muchos del modelo es Rol — Permiso.

9.6 Estados del usuario y de la mesa

`usuarios.estado` (ACTIVO o INACTIVO) permite al administrador inactivar o reactivar cuentas, incluidas las de clientes, sin borrarlas. Un usuario INACTIVO no puede iniciar sesión (HU-09) y conserva su historial de reservas (HU-11, RN-08). Como las claves foráneas de `reservas` y `registros_cambio` usan RESTRICT, inactivar es la forma normal de dar de baja a un cliente. No se confunde con la reserva activa (estado PENDIENTE o CONFIRMADA, H-3).

`mesas.estado` distingue el estado operativo (LIBRE, RESERVADA, OCUPADA) de la mesa fuera de servicio (INACTIVA, MANTENIMIENTO). Una mesa fuera de servicio no se ofrece en la consulta de disponibilidad (RF-03) ni recibe reservas nuevas.

9.7 Coherencia con las reglas de negocio

No todas las reglas pueden garantizarse con claves y restricciones. La tabla separa lo que asegura la base de datos de lo que necesita lógica adicional.

Regla	Cómo la representa el modelo	¿La garantiza la base de datos?	Qué se necesita además
Reserva asociada a cliente y mesa (HU-04, RF-04)	id_cliente e id_mesa NN con FK	Sí, en cuanto a que existan el usuario y la mesa	Verificar que el usuario tenga rol Cliente y estado ACTIVO
No cruce de reservas activas (RN-01, RF-09, RF-10, RNF-04)	La reserva guarda id_mesa, fecha, hora_inicio, hora_fin y estado	**No** por sí sola. Un UNIQUE sobre (id_mesa, fecha, hora_inicio) solo detectaría inicios idénticos, no cruces parciales (19:00–21:00 contra 20:00–22:00), e impediría reservar una mesa cuya reserva anterior fue cancelada	Transacción con bloqueo de fila (SELECT … FOR UPDATE) o trigger que busque cruces con reservas PENDIENTE o CONFIRMADA de la misma mesa. Aplica a crear, modificar (RN-09), reubicar y volver a reservar
Capacidad suficiente (RN-02)	numero_personas y mesas.capacidad	Parcial: los CK de valores positivos son locales; un CK no compara columnas de dos tablas	Trigger o validación en la transacción de crear, modificar o reubicar
Mesa fuera de servicio (H-9)	mesas.estado con INACTIVA y MANTENIMIENTO	Solo que el valor sea válido	La consulta de disponibilidad y la validación de reserva excluyen esas mesas; al sacar una mesa de servicio, sus reservas activas futuras se reubican (D-5)

Regla	Cómo la representa el modelo	¿La garantiza la base de datos?	Qué se necesita además
Correo y teléfono únicos (RN-04, RN-14)	UK en usuarios.correo y usuarios.telefono	Sí, también al editar el perfil	Validar formato (HU-01) y normalizar el correo en la aplicación
Número de mesa único y capacidad mayor a cero (RN-03, HU-02)	UK en mesas.numero; CK en capacidad	Sí	No aplica
Código de reserva único y no modificable (RF-05)	UK en codigo_unico	La unicidad sí; la inmutabilidad no	Generar el código en el servicio y bloquear su modificación (trigger BEFORE UPDATE o permisos)
Reserva cancelada no se reactiva (RN-07, RN-08)	La reserva no se elimina y conserva el estado CANCELADA	Conservar el estado sí; prohibir la transición CANCELADA → CONFIRMADA no	Trigger o lógica de servicio; volver a reservar equivale a crear una reserva nueva con la validación completa
Anticipación mínima y estado inicial (RN-05, RN-06)	configuracion_restaurante; ENUM en estado_inicial_reserva	Solo que el estado inicial sea PENDIENTE o CONFIRMADA	El servicio compara fecha y hora con la anticipación y asigna el estado inicial leído de la fila única
Cambios de rol y permisos con responsable, fecha y hora (RN-12, RF-14)	registros_cambio con FK NN al responsable, fecha_hora NN y descripcion NN	Sí: que cada registro tenga responsable existente, fecha y descripción. No: que todo cambio genere su registro	Guardar el cambio y su registro en la misma transacción (o con trigger)

Regla	Cómo la representa el modelo	¿La garantiza la base de datos?	Qué se necesita además
Cuenta inactiva o bloqueada (HU-09, H-4, H-5)	usuarios.estado, intentos_fallidos, bloqueado_hasta	Solo almacena los datos	El inicio de sesión rechaza estado INACTIVO o bloqueado_hasta futuro, incrementa el contador al fallar y lo reinicia al acertar
Estado de mesa actualizado (HU-02, RNF-02)	mesas.estado	Solo que el valor sea válido	Actualizarlo en la misma transacción que modifica la reserva
Un cliente ve solo sus reservas (RN-10)	reservas.id_cliente permite filtrar	No (es control de acceso)	Autorización en la aplicación según rol y permisos

Un usuario, un solo rol (RF-15, HU-09, RN-13). La columna `usuarios.id_rol` es NN y es una sola, así que la base de datos garantiza por estructura que cada usuario tiene exactamente un rol y no puede tener dos. El inicio de sesión carga ese único rol y sus permisos; para ingresar con otro rol hace falta que el administrador cambie el rol del usuario, y ese cambio queda en `registros_cambio` (RN-12).

En resumen, la base de datos garantiza las relaciones, las unicidades y los valores válidos. Las reglas que comparan varias filas o tablas (cruce de horarios, capacidad, transiciones de estado y trazabilidad de cambios) dependen de transacciones, triggers o lógica de servicio, que se implementan en el script SQL y en la aplicación.

9.8 Notas de implementación en MySQL

- MySQL no tiene objetos SEQUENCE: la numeración del código de reserva (formato propuesto RES-AAAAMMDD-#####) se resuelve en el script con una tabla auxiliar. Es un detalle de implementación y no hace parte del modelo lógico.
- La fila única de `configuracion_restaurante` se asegura con CK id_configuracion = 1.
- El token temporal de recuperación de contraseña (RF-20) se define en la implementación (H-4).

9.9 Verificación de consistencia

Cada tabla del modelo lógico corresponde a una entidad del MER (más `roles_permisos`, que resuelve la relación N:M) y coincide con el resultado de la normalización (7.7). Todas las claves foráneas de la sección 5 apuntan a claves primarias existentes: roles(id_rol), usuarios(id_usuario), mesas(id_mesa), reservas(id_reserva) y permisos(id_permiso). Las cardinalidades coinciden con las reglas de negocio documentadas.

10. Trazabilidad con los requisitos

Los códigos corresponden a las Fases 1, 2 y 3.

Entidad o relación	Historia, requisito o regla relacionada	Justificación de su inclusión
USUARIO	HU-01, HU-09, HU-10, HU-12; RF-01, RF-15, RF-16, RF-17, RF-19, RF-20; RN-04, RN-13, RN-14	Datos de registro, contacto y acceso; unicidad de correo y teléfono; bloqueo temporal y estado ACTIVO o INACTIVO
ROL	HU-08; RF-13; RN-10; RT-04	Representa los roles cliente, recepcionista y administrador
PERMISO y ROL_PERMISO	HU-08; RF-13; RNF-03	Un mismo permiso puede estar en varios roles
MESA	HU-02, HU-03; RF-02, RF-03; RN-02, RN-03	Número único, capacidad, ubicación y estado, incluidas las mesas fuera de servicio
RESERVA	HU-04 a HU-07, HU-11; RF-04, RF-05, RF-07 a RF-12, RF-18; RN-01, RN-06 a RN-09, RN-11; RNF-02, RNF-04	Núcleo del sistema: código, fecha, horario, personas y estado
NOTIFICACION	HU-04; RF-06	Notificación al cliente al crear la reserva

Entidad o relación	Historia, requisito o regla relacionada	Justificación de su inclusión
REGISTRO_CAMBIO	HU-08; RF-14; RN-12	Responsable, afectado, fecha y descripción de cada cambio de rol o permiso
CONFIGURACION_RESTAURANTE	HU-04; RF-05; RN-05, RN-06	Anticipación mínima y estado inicial; sin HU ni RF propia (Fase 3, §4.4 y §5.1; RT-05)
ROL — USUARIO	RF-13, RF-15; RN-13	Cada usuario inicia sesión con un rol que define sus permisos
USUARIO — RESERVA	HU-04, HU-11; RF-18; RN-10	Asocia cada reserva a su cliente y permite el historial
MESA — RESERVA	HU-06; RF-09, RF-10; RN-01, RN-02	Permite validar cruces y capacidad
RESERVA — NOTIFICACION	RF-06	Vincula la notificación con su reserva
USUARIO — REGISTRO_CAMBIO	HU-08; RF-14; RN-12	Identifica al responsable y, si aplica, al usuario afectado

11. Conclusiones

- El modelo se construyó con los nombres, atributos y relaciones de la Fase 3, sin agregar módulos fuera del alcance. Resultan nueve tablas en tercera forma normal para MySQL 8.0, con nombres en plural.
- La normalización mostró dependencias reales: los datos de cliente, mesa y configuración mezclados con la reserva, y los permisos agrupados dentro de los roles. Las demás estructuras ya cumplían las tres formas normales.
- Se tomaron decisiones sobre diferencias con la Fase 3: un solo rol por usuario, Rol — Permiso muchos a muchos, herencia en tabla única, asociación confirma/reubica sin tabla propia, y las extensiones de estado del usuario (ACTIVO o INACTIVO), control de

intentos de inicio de sesión, usuario afectado en el registro de cambios, mesas fuera de servicio (INACTIVA, MANTENIMIENTO) y configuración del restaurante. Todas están justificadas y pueden revertirse sin afectar el resto.

- La base de datos garantiza las relaciones, la unicidad de correo, teléfono, número de mesa y código de reserva, y los valores válidos de estado. No garantiza por sí sola la ausencia de cruces de reservas ni la capacidad de la mesa: esas reglas (RN-01, RN-02, RF-10) requieren transacciones con bloqueo, triggers o lógica de servicio.
- La trazabilidad relaciona cada tabla y relación con historias de usuario, requisitos y reglas de negocio, lo que facilita sustentar el modelo ante el instructor.